Reporte Técnico de Evidencias

Taller Práctico Kubernetes — Edición Enterprise Extended (Módulos 3 a 8)

Alumno	Gilson Ponce Briones
Namespace	`gponceb`
Fecha de entrega	23 de octubre de 2026
Asignatura	Administración Avanzada de Contenedores y Orquestación con Kubernetes
Repositorio	`https://github.com/GilsonPonce/taller-k8s-gponceb`
Entorno	Kubernetes v1.37.1 (kubeadm, 1 nodo) · containerd 2.4.2 · Calico · ingress-nginx v1.15.1 · metrics-server v0.9.0 · local-path

1. Resumen

Se desplegó la arquitectura completa del taller en el namespace `gponceb`: **Ingress NGINX → Frontend (NGINX estático) → Backend Node.js** (InitContainer de verificación de BD + API + sidecar de logs) **→ PostgreSQL StatefulSet** con volumen persistente, más HPA, DaemonSet de logging, Job de migración, CronJob de backups sobre un PVC dedicado, PodDisruptionBudget, ResourceQuota/LimitRange y aislamiento Zero-Trust con NetworkPolicies. Todos los pasos de la guía se ejecutaron y verificaron; la salida real de cada comando está en `evidencias/*.txt`.

Verificación	Resultado
Pods, Services, HPA y StatefulSet	**OK** todos Running / Ready
CRUD web en `http://localhost`	**OK** listar, crear y eliminar desde la interfaz
HPA bajo carga	**OK** 2 → 4 → 8 → 10 réplicas deseadas (CPU 452%/70%); la ResourceQuota limitó a 5 pods reales; regresó a 2 al terminar la carga
CronJob de backup + PVC	**OK** ejecución programada (15:05) y manuales; 3 archivos `.sql` en `backup-pvc`
Zero-Trust	**OK** pod no autorizado bloqueado hacia backend y BD; app y backups siguen funcionando
Troubleshooting (6 escenarios)	**OK** reproducidos, diagnosticados y resueltos

2. Adaptaciones y correcciones a la guía

La guía está escrita para Docker Desktop. Al ejecutarla tal cual se detectaron fallos; cada corrección está comentada en el manifiesto correspondiente.

Ítem	Guía	Problema	Solución aplicada
Entorno	Docker Desktop (Windows)	Clúster kubeadm de 1 nodo (Arch Linux, containerd 2.4, K8s v1.37.1)	Se instalaron los componentes que Docker Desktop trae de serie: `cluster-setup/install-addons.sh`.
CNI	—	flannel no aplica NetworkPolicies	Reemplazo por Calico v3.32.2 (mismo podSubnet 10.244.0.0/16).
Imágenes	`docker build` visible por K8s	containerd no ve las imágenes de Docker	`scripts/build-images.sh`: build + `docker save | ctr -n k8s.io images import`.
Dockerfile backend	`npm ci --only=production`	`npm ci` exige `package-lock.json` (no existía)	Se generó y versionó `package-lock.json`; flag actualizado a `--omit=dev`.
04-ingress-nginx	`rewrite-target: /$2`	Quita `/api` y el backend responde 404 (rutas `/api/productos`)	Paths `Prefix` `/api` y `/` sin rewrite.
Ingress Controller	manifiesto *cloud* v1.8.2	Sin LoadBalancer en bare-metal; v1.8.2 no soporta K8s 1.37	ingress-nginx v1.15.1 (baremetal) con `hostPort` 80/443 → `http://localhost`.
HPA	metrics-server	No instalado; certificados del kubelet autofirmados	metrics-server v0.9.0 con `--kubelet-insecure-tls` (solo laboratorio).
02-backup-pvc	`ReadWriteMany`	local-path (y hostpath de Docker Desktop) no soportan RWX → Pending	`ReadWriteOnce` (1 nodo: CronJob e inspección lo montan sin problema).
NetworkPolicies	05/06/07	Con default-deny: el Ingress Controller no llegaba al backend, PostgreSQL rechazaba todo y los Jobs no tenían DNS	07 permite el namespace `ingress-nginx`; nuevo `08-database-networkpolicy.yaml` (BD ← backend y `role=db-client`); DNS TCP+UDP.
ResourceQuota	02-namespace-quota	Con cuota de CPU/memoria se rechazan pods sin requests/limits (pods de prueba, Módulo 8)	Nuevo `03-limitrange.yaml` con valores por defecto; requests/limits en init/sidecar/postgres/jobs.
Mod 4 probes	Deployment sin sidecar	Aplicarlo eliminaba el sidecar y el ConfigMap del Módulo 3	El archivo del Módulo 4 conserva sidecar y `envFrom`.
Paso 8	`kubectl run ... -v backup-pvc:/backups`	En kubectl `-v` es verbosidad, no volumen	`scripts/inspect-backups.sh` monta el PVC con `--overrides`.
Archivos faltantes	04-backend-configmap, 03-statefulset-volume-claim-template	Listados en la estructura sin contenido	Creados y documentados.
01-db-pvc	`postgres-pvc`	El StatefulSet usa `volumeClaimTemplates`; este PVC no lo consume nadie y queda Pending (WaitForFirstConsumer)	Se mantiene por fidelidad a la guía; documentado.

3. Evidencia obligatoria 1 — `kubectl get all -n gponceb`

gponce@dev: ~/taller-k8s-gponceb — kubectl get all -n gponceb

```
$ kubectl get all -n gponceb
NAME                                    READY   STATUS      RESTARTS   AGE
pod/backend-api-7f8496c7f9-68znx        2/2     Running     0          20m
pod/backend-api-7f8496c7f9-rbtbx        2/2     Running     0          20m
pod/backup-manual-1-gcxq5               0/1     Completed   0          5m32s
pod/backup-manual-2-rg82b               0/1     Completed   0          4m24s
pod/db-backup-cronjob-29879465-8hthm    0/1     Completed   0          116s
pod/db-migration-job-dzfdm              0/1     Completed   0          5m35s
pod/fluentd-logging-q4t9k               1/1     Running     0          37m
pod/frontend-web-c87d4b684-brc6n        1/1     Running     0          20m
pod/frontend-web-c87d4b684-cz4q6        1/1     Running     0          20m
pod/postgres-db-0                       1/1     Running     0          37m

NAME                       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
service/backend-service    ClusterIP   10.96.140.224    <none>        5678/TCP       37m
service/frontend-service   NodePort    10.100.189.225   <none>        80:30080/TCP   37m
service/postgres-service   ClusterIP   10.97.136.121    <none>        5432/TCP       37m

NAME                              DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR
AGE
daemonset.apps/fluentd-logging    1         1         1       1            1           <none>
37m

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend-api    2/2     2            2           20m
deployment.apps/frontend-web   2/2     2            2           20m

NAME                                      DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-api-58f6b74679    0         0         0       20m
replicaset.apps/backend-api-7f8496c7f9    2         2         2       20m
replicaset.apps/frontend-web-c87d4b684    2         2         2       20m

NAME                            READY   AGE
statefulset.apps/postgres-db    1/1     37m

NAME                                                REFERENCE                 TARGETS       MINPODS   MAX
PODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/backend-hpa   Deployment/backend-api    cpu: 1%/70%   2         10
2          20m

NAME                                SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
cronjob.batch/db-backup-cronjob   */5 * * * *   <none>     False     0        116s            5m32s

NAME                                   STATUS     COMPLETIONS   DURATION   AGE
job.batch/backup-manual-1              Complete   1/1           7s         5m32s
job.batch/backup-manual-2              Complete   1/1           3s         4m24s
job.batch/db-backup-cronjob-29879465   Complete   1/1           3s         116s
job.batch/db-migration-job             Complete   1/1           10s        5m42s

$ kubectl get pvc,ingress,networkpolicy,pdb,resourcequota,limitrange,configmap,secret -n gponceb
NAME                                                  STATUS    VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/backup-pvc                      Bound     pvc-728907df-f256-4063-83cc-7e5158365
f72   5Gi        RWO            local-path     <unset>                 37m
persistentvolumeclaim/postgres-pvc                    Pending
local-path     <unset>                 37m
persistentvolumeclaim/postgres-storage-postgres-db-0   Bound     pvc-83ecc35a-07d7-43ee-afaa-0c0c23e86
a20   2Gi        RWO            local-path     <unset>                 37m

NAME                                    CLASS   HOSTS   ADDRESS       PORTS   AGE
ingress.networking.k8s.io/app-ingress   nginx   *       10.51.20.21   80      19m
```

```
NAME                                              POD-SELECTOR       AGE
networkpolicy.networking.k8s.io/allow-backend     app=backend-api    61s
networkpolicy.networking.k8s.io/allow-database    app=postgres       4m44s
networkpolicy.networking.k8s.io/allow-db-clients  role=db-client     4m44s
networkpolicy.networking.k8s.io/allow-frontend    app=frontend-web   4m44s
networkpolicy.networking.k8s.io/default-deny-all  <none>             4m44s

NAME                                    MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/backend-pdb  1               N/A               1                     20m

NAME                          REQUEST                                                      LIMIT
AGE
resourcequota/compute-quota   pods: 6/20, requests.cpu: 520m/2, requests.memory: 772Mi/2Gi   limits.cp
u: 2300m/4, limits.memory: 1544Mi/4Gi   20m

NAME                         CREATED AT
limitrange/default-limits    2026-10-23T14:46:47Z

NAME                          DATA   AGE
configmap/backend-config      4      37m
configmap/db-init-script      1      37m
configmap/kube-root-ca.crt    1      37m

NAME                TYPE     DATA   AGE
secret/db-secret    Opaque   3      37m

$ kubectl top pods -n gponceb
NAME                           CPU(cores)   MEMORY(bytes)
backend-api-7f8496c7f9-68znx   1m           25Mi
backend-api-7f8496c7f9-rbtbx   1m           24Mi
fluentd-logging-q4t9k          1m           64Mi
frontend-web-c87d4b684-brc6n   0m           22Mi
frontend-web-c87d4b684-cz4q6   0m           22Mi
postgres-db-0                  1m           29Mi
```

4. Evidencia obligatoria 2 — Interfaz web en `http://localhost`

Capturas tomadas con un navegador (Chromium headless) a través del Ingress Controller. La interfaz consume la API en `/api/productos`, que el Ingress enruta a `backend-service:5678`.

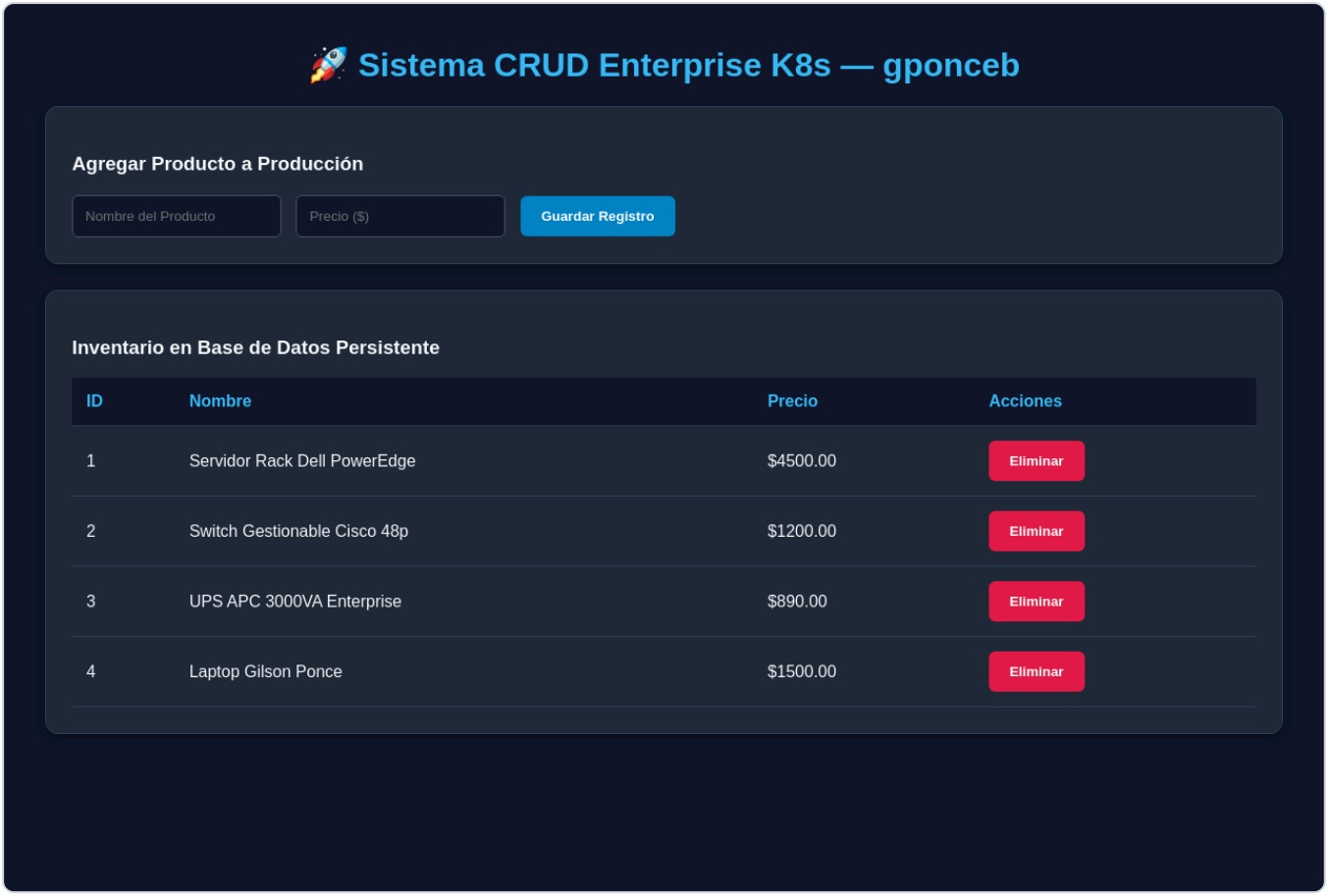


Fig. 1 — Listado inicial: datos de `init.sql` + producto creado vía API (id 4).

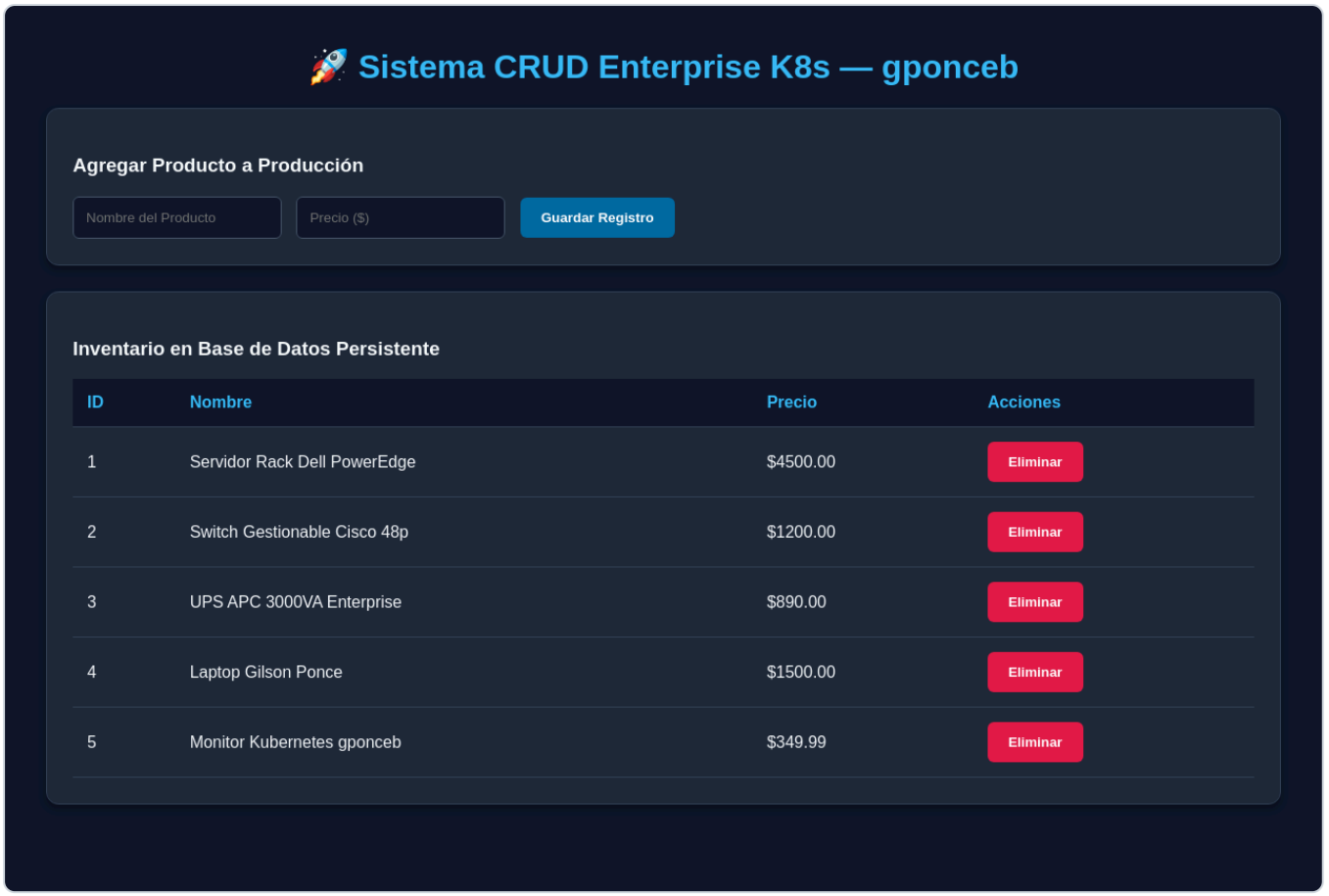


Fig. 2 — Crear: se registra “Monitor Kubernetes gponceb” desde el formulario (id 5).

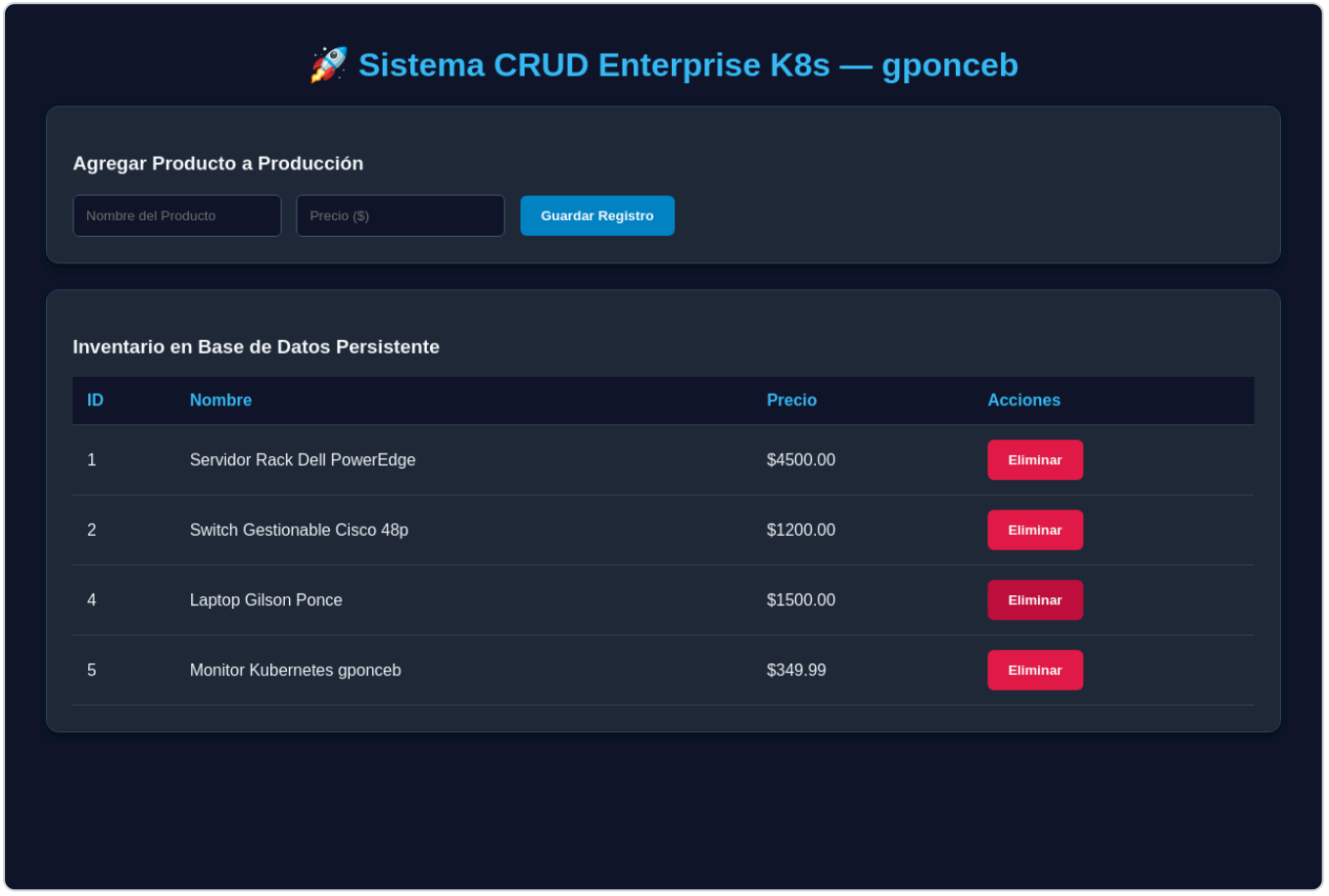


Fig. 3 — Eliminar: se borra “UPS APC 3000VA Enterprise” (id 3) con el botón Eliminar.

gponce@dev: ~/taller-k8s-gponceb — Ingress y API vía curl

```
$ kubectl apply -f mod5-networking/04-ingress-nginx.yaml
ingress.networking.k8s.io/app-ingress created

$ kubectl get ingress
NAME          CLASS   HOSTS   ADDRESS   PORTS   AGE
app-ingress   nginx   *                 80      10s

$ curl -s -o /dev/null -w 'GET / -> HTTP %{http_code}
' http://localhost/
GET / -> HTTP 200

$ curl -s http://localhost/api/productos
[{"id":1,"nombre":"Servidor Rack Dell PowerEdge","precio":"4500.00","creado_en":"2026-10-23T14:29:39.0
37Z"},{"id":2,"nombre":"Switch Gestionable Cisco 48p","precio":"1200.00","creado_en":"2026-10-23T14:2
9:39.037Z"},{"id":3,"nombre":"UPS APC 3000VA Enterprise","precio":"890.00","creado_en":"2026-10-23T14:
29:39.037Z"}]
$ curl -s -X POST -H 'Content-Type: application/json' -d '{"nombre":"Laptop Gilson Ponce","precio":150
0}' http://localhost/api/productos
{"id":4,"nombre":"Laptop Gilson Ponce","precio":"1500.00","creado_en":"2026-10-23T14:47:42.621Z"}
```

5. Evidencia obligatoria 3 — CronJob y respaldos en `backup-pvc`

El Job de migración insertó “Audífonos Canceling Sony”. El CronJob (`*/5 * * * *`) se ejecutó de forma programada (`db-backup-cronjob-29879465`, 15:05) y además se lanzaron dos ejecuciones manuales con `kubectl create job --from=cronjob`, la segunda ya con Zero-Trust activo. El respaldo más reciente refleja el estado del CRUD (sin el id 3 eliminado).

gponce@dev: ~/taller-k8s-gponceb — Job, CronJob y contenido de backup-pvc

```
$ kubectl apply -f mod3-workloads/08-db-migration-job.yaml
job.batch/db-migration-job created

$ kubectl get job db-migration-job
NAME               STATUS     COMPLETIONS   DURATION   AGE
db-migration-job   Complete   1/1           10s        10s

$ kubectl logs job/db-migration-job
INSERT 0 1

$ kubectl apply -f mod3-workloads/09-db-backup-cronjob-pvc.yaml
cronjob.batch/db-backup-cronjob created

$ kubectl create job backup-manual-1 --from=cronjob/db-backup-cronjob
job.batch/backup-manual-1 created

$ kubectl logs job/backup-manual-1
=== GENERANDO BACKUP EN VOLUMEN PERSISTENTE ===
Backup guardado exitosamente en: /backups/backup-20261023-150129.sql

$ kubectl get pvc backup-pvc
NAME         STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLAS
S   VOLUMEATTRIBUTESCLASS   AGE
backup-pvc   Bound    pvc-728907df-f256-4063-83cc-7e5158365f72   5Gi        RWO            local-path
<unset>                 32m

$ bash scripts/inspect-backups.sh
pod/inspect-backup created
total 4
-rw-r--r--    1 root     root          2488 Oct 23 15:01 backup-20261023-150129.sql

Datos del respaldo mas reciente:
COPY public.productos (id, nombre, precio, creado_en) FROM stdin;
1       Servidor Rack Dell PowerEdge    4500.00 2026-10-23 14:29:39.037578
2       Switch Gestionable Cisco 48p    1200.00 2026-10-23 14:29:39.037578
4       Laptop Gilson Ponce     1500.00 2026-10-23 14:47:42.621505
5       Monitor Kubernetes gponceb      349.99  2026-10-23 14:56:04.195915
6       Audífonos Canceling Sony        320.00  2026-10-23 15:01:22.300395
\.


$ kubectl get cronjob db-backup-cronjob
NAME                SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
db-backup-cronjob   */5 * * * *   <none>     False     0        93s             5m9s

$ kubectl get jobs -o custom-columns=JOB:.metadata.name,CREADO_POR:.metadata.annotations.cronjob\.kube
rnetes\.io/instantiate,COMPLETIONS:.status.succeeded,INICIO:.status.startTime
JOB                          CREADO_POR   COMPLETIONS   INICIO
backup-manual-1              manual       1             2026-10-23T15:01:24Z
backup-manual-2              manual       1             2026-10-23T15:02:32Z
db-backup-cronjob-29879465   <none>       1             2026-10-23T15:05:00Z
db-migration-job             <none>       1             2026-10-23T15:01:14Z

$ bash scripts/inspect-backups.sh
```

```
pod/inspect-backup created
total 12
-rw-r--r--    1 root     root          2488 Oct 23 15:01 backup-20261023-150129.sql
-rw-r--r--    1 root     root          2488 Oct 23 15:02 backup-20261023-150233.sql
-rw-r--r--    1 root     root          2488 Oct 23 15:05 backup-20261023-150500.sql

Datos del respaldo mas reciente:
COPY public.productos (id, nombre, precio, creado_en) FROM stdin;
1       Servidor Rack Dell PowerEdge    4500.00 2026-10-23 14:29:39.037578
2       Switch Gestionable Cisco 48p    1200.00 2026-10-23 14:29:39.037578
4       Laptop Gilson Ponce     1500.00 2026-10-23 14:47:42.621505
5       Monitor Kubernetes gponceb      349.99  2026-10-23 14:56:04.195915
6       Audífonos Canceling Sony        320.00  2026-10-23 15:01:22.300395
\.
```

6. Evidencias complementarias por paso

Pasos 1–3: Namespace, almacenamiento, secretos y PostgreSQL

gponce@dev: ~/taller-k8s-gponceb — 01-namespace

```
$ kubectl apply -f 00-namespace.yaml
namespace/gponceb created

$ kubectl config set-context --current --namespace=gponceb
Context "kubernetes-admin@kubernetes" modified.

$ kubectl config view --minify | grep namespace:
    namespace: gponceb
```

gponce@dev: ~/taller-k8s-gponceb — 02-storage-secrets

```
$ kubectl apply -f mod6-storage/01-db-pvc.yaml
persistentvolumeclaim/postgres-pvc created

$ kubectl apply -f mod6-storage/02-backup-pvc.yaml
persistentvolumeclaim/backup-pvc created

$ kubectl apply -f mod3-workloads/01-db-secret.yaml
secret/db-secret created

$ kubectl apply -f mod3-workloads/02-db-init-configmap.yaml
configmap/db-init-script created

$ kubectl apply -f mod3-workloads/04-backend-configmap.yaml
configmap/backend-config created
```

gponce@dev: ~/taller-k8s-gponceb — 03-database

```
$ kubectl apply -f mod3-workloads/03-db-statefulset.yaml
statefulset.apps/postgres-db created

$ kubectl apply -f mod5-networking/01-db-service.yaml
service/postgres-service created

$ kubectl get pods -l app=postgres -o wide
NAME            READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED NODE   READINESS G
ATES
postgres-db-0   1/1     Running   0          12s   10.244.90.138   dev    <none>           <none>

$ kubectl get pvc
NAME                             STATUS    VOLUME                                     CAPACITY   ACCES
S MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
backup-pvc                       Pending
local-path     <unset>                 12s
postgres-pvc                     Pending
local-path     <unset>                 12s
postgres-storage-postgres-db-0   Bound     pvc-83ecc35a-07d7-43ee-afaa-0c0c23e86a20   2Gi        RWO
local-path     <unset>                 12s

$ kubectl logs postgres-db-0 | grep -E 'init.sql|INSERT|init process complete|ready to accept' | tail
-5
2026-10-23 14:29:38.882 UTC [48] LOG:  database system is ready to accept connections
/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/init.sql
INSERT 0 3
PostgreSQL init process complete; ready for start up.
2026-10-23 14:29:39.408 UTC [1] LOG:  database system is ready to accept connections
```

```
$ kubectl exec postgres-db-0 -- psql -U admin_user -d tienda_db -c 'SELECT id,nombre,precio FROM produ
ctos'
 id |            nombre            | precio
----+------------------------------+---------
  1 | Servidor Rack Dell PowerEdge | 4500.00
  2 | Switch Gestionable Cisco 48p | 1200.00
  3 | UPS APC 3000VA Enterprise    |  890.00
(3 rows)
```

Paso 4: Workloads multicontenedor (InitContainer + sidecar)

gponce@dev: ~/taller-k8s-gponceb — 04-workloads

```
$ kubectl apply -f mod5-networking/02-backend-service.yaml
service/backend-service created

$ kubectl apply -f mod5-networking/03-frontend-service.yaml
service/frontend-service created

$ kubectl apply -f mod3-workloads/07-fluentd-daemonset.yaml
daemonset.apps/fluentd-logging created

$ kubectl apply -f mod3-workloads/05-backend-deployment-multicontainer.yaml
deployment.apps/backend-api created

$ kubectl apply -f mod3-workloads/06-frontend-deployment-sidecar.yaml
deployment.apps/frontend-web created

$ kubectl get pods -o wide
NAME                            READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED NOD
E   READINESS GATES
backend-api-58f6b74679-j6z7l    2/2     Running   0          5s    10.244.90.141   dev    <none>
<none>
backend-api-58f6b74679-thb2w    2/2     Running   0          5s    10.244.90.140   dev    <none>
<none>
fluentd-logging-q4t9k           1/1     Running   0          16m   10.244.90.139   dev    <none>
<none>
frontend-web-c87d4b684-brc6n    1/1     Running   0          5s    10.244.90.142   dev    <none>
<none>
frontend-web-c87d4b684-cz4q6    1/1     Running   0          5s    10.244.90.143   dev    <none>
<none>
postgres-db-0                   1/1     Running   0          16m   10.244.90.138   dev    <none>
<none>

$ kubectl logs deploy/backend-api -c check-db-ready
Found 2 pods, using pod/backend-api-58f6b74679-thb2w

$ kubectl logs deploy/backend-api -c api
Found 2 pods, using pod/backend-api-58f6b74679-thb2w
Backend API escuchando en el puerto 5678
```

Módulos 4 y 7: Probes, PDB, LimitRange y ResourceQuota

gponce@dev: ~/taller-k8s-gponceb — 05-probes-hardening

```
$ kubectl apply -f mod4-config/01-production-probes-resources.yaml
deployment.apps/backend-api configured

$ kubectl describe deploy backend-api | grep -E 'Liveness|Readiness|Containers:|^   [a-z-]+:$'
  Init Containers:
   check-db-ready:
  Containers:
   api:
    Liveness:   http-get http://:5678/healthz delay=15s timeout=1s period=10s successThreshold=1 failu
reThreshold=3
```

```
    Readiness:  http-get http://:5678/healthz delay=5s timeout=1s period=5s successThreshold=1 failure
Threshold=3
   log-adapter-sidecar:
   shared-logs:

$ kubectl apply -f mod7-production-hardening/01-backend-pdb.yaml
poddisruptionbudget.policy/backend-pdb created

$ kubectl apply -f mod7-production-hardening/03-limitrange.yaml
limitrange/default-limits created

$ kubectl apply -f mod7-production-hardening/02-namespace-quota.yaml
resourcequota/compute-quota created

$ kubectl get pdb
NAME          MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
backend-pdb   1               N/A               1                     1s

$ kubectl describe resourcequota compute-quota
Name:            compute-quota
Namespace:       gponceb
Resource         Used    Hard
--------         ----    ----
limits.cpu       3400m   4
limits.memory    2120Mi  4Gi
pods             8       20
requests.cpu     740m    2
requests.memory  1060Mi  2Gi
```

Paso 5: Autoscaling HPA bajo carga

Carga generada desde el host con `scripts/load-test.js` (64 conexiones concurrentes durante 240 s, ≈1,4 millones de peticiones sin errores). El HPA escaló 2 → 4 → 8 → 10 réplicas deseadas; la **ResourceQuota** (`limits.cpu: 4`) rechazó los pods adicionales (*exceeded quota*), dejando 5 réplicas reales — interacción esperada entre autoscaling y cuotas. Tras la carga el HPA volvió a 2 réplicas (`minReplicas`).

```
gponce@dev: ~/taller-k8s-gponceb — 06-hpa

$ kubectl apply -f mod3-workloads/10-hpa-autoscaler.yaml
horizontalpodautoscaler.autoscaling/backend-hpa created

$ kubectl get hpa
NAME          REFERENCE                TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend-api   cpu: 2%/70%   2         10        2          45s

$ kubectl top pods
NAME                           CPU(cores)   MEMORY(bytes)
backend-api-7f8496c7f9-68znx   2m           19Mi
backend-api-7f8496c7f9-rbtbx   2m           19Mi
fluentd-logging-q4t9k          1m           64Mi
frontend-web-c87d4b684-brc6n   0m           22Mi
frontend-web-c87d4b684-cz4q6   0m           22Mi
postgres-db-0                  1m           25Mi

$ node scripts/load-test.js 240 64   # carga desde el host a http://localhost/api/productos
--- t=15s
backend-hpa   Deployment/backend-api   cpu: 12%/70%   2     10    2     4m39s
--- t=30s
backend-hpa   Deployment/backend-api   cpu: 59%/70%   2     10    2     4m54s
--- t=45s
backend-hpa   Deployment/backend-api   cpu: 414%/70%   2     10    2     5m10s
--- t=60s
backend-hpa   Deployment/backend-api   cpu: 452%/70%   2     10    4     5m25s
--- t=75s
```

```
backend-hpa   Deployment/backend-api   cpu: 419%/70%   2     10    8     5m40s
--- t=90s
backend-hpa   Deployment/backend-api   cpu: 383%/70%   2     10    10    5m55s
--- t=105s
backend-hpa   Deployment/backend-api   cpu: 350%/70%   2     10    10    6m10s
--- t=120s
backend-hpa   Deployment/backend-api   cpu: 356%/70%   2     10    10    6m25s
--- t=135s
backend-hpa   Deployment/backend-api   cpu: 359%/70%   2     10    10    6m40s
--- t=150s
backend-hpa   Deployment/backend-api   cpu: 357%/70%   2     10    10    6m55s
--- t=165s
backend-hpa   Deployment/backend-api   cpu: 358%/70%   2     10    10    7m10s
--- t=180s
backend-hpa   Deployment/backend-api   cpu: 362%/70%   2     10    10    7m25s
--- t=195s
backend-hpa   Deployment/backend-api   cpu: 356%/70%   2     10    10    7m40s
--- t=210s
backend-hpa   Deployment/backend-api   cpu: 351%/70%   2     10    10    7m55s
--- t=225s
backend-hpa   Deployment/backend-api   cpu: 345%/70%   2     10    10    8m10s
--- t=240s
backend-hpa   Deployment/backend-api   cpu: 342%/70%   2     10    10    8m26s
peticiones OK=1411042 fallidas=0
FIN: OK=1411064 fallidas=0
$ kubectl get pods -l app=backend-api
NAME                           READY   STATUS    RESTARTS   AGE
backend-api-7f8496c7f9-68znx   2/2     Running   0          8m41s
backend-api-7f8496c7f9-fgjjt   2/2     Running   0          3m26s
backend-api-7f8496c7f9-hvsd9   2/2     Running   0          3m26s
backend-api-7f8496c7f9-k8ffl   2/2     Running   0          3m11s
backend-api-7f8496c7f9-rbtbx   2/2     Running   0          8m34s

$ kubectl describe hpa backend-hpa | sed -n '/Events:/,$p'
Events:
  Type     Reason                        Age    From                       Message
  ----     ------                        ----   ----                       -------
  Warning  FailedGetResourceMetric       8m26s  horizontal-pod-autoscaler  failed to get cpu utilizati
on: did not receive metrics for targeted pods (pods might be unready)
  Warning  FailedComputeMetricsReplicas  8m26s  horizontal-pod-autoscaler  invalid metrics (1 invalid
out of 1), first error is: failed to get cpu resource metric value: failed to get cpu utilization: did
not receive metrics for targeted pods (pods might be unready)
  Normal   SuccessfulRescale             3m26s  horizontal-pod-autoscaler  New size: 4; reason: cpu re
source utilization (percentage of request) above target
  Normal   SuccessfulRescale             3m11s  horizontal-pod-autoscaler  New size: 8; reason: cpu re
source utilization (percentage of request) above target
  Normal   SuccessfulRescale             2m56s  horizontal-pod-autoscaler  New size: 10; reason: cpu r
esource utilization (percentage of request) above target

$ kubectl describe resourcequota compute-quota | grep -E 'limits.cpu|pods'
limits.cpu       3950m   4
pods             9       20

$ kubectl get events --field-selector reason=FailedCreate | tail -3
3m10s       Warning   FailedCreate   replicaset/backend-api-7f8496c7f9   Error creating: pods "backend
-api-7f8496c7f9-wck4z" is forbidden: exceeded quota: compute-quota, requested: limits.cpu=550m, used:
limits.cpu=3950m, limited: limits.cpu=4
3m10s       Warning   FailedCreate   replicaset/backend-api-7f8496c7f9   Error creating: pods "backend
-api-7f8496c7f9-v8tcz" is forbidden: exceeded quota: compute-quota, requested: limits.cpu=550m, used:
limits.cpu=3950m, limited: limits.cpu=4
2m56s       Warning   FailedCreate   replicaset/backend-api-7f8496c7f9   (combined from similar event
s): Error creating: pods "backend-api-7f8496c7f9-fdmp8" is forbidden: exceeded quota: compute-quota, r
equested: limits.cpu=550m, used: limits.cpu=3950m, limited: limits.cpu=4

$ kubectl get hpa backend-hpa
NAME          REFERENCE                TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend-api   cpu: 1%/70%   2         10        2          14m

$ kubectl describe hpa backend-hpa | grep SuccessfulRescale
```

```
  Normal   SuccessfulRescale            9m27s  horizontal-pod-autoscaler  New size: 4; reason: cpu re
source utilization (percentage of request) above target
  Normal   SuccessfulRescale            9m12s  horizontal-pod-autoscaler  New size: 8; reason: cpu re
source utilization (percentage of request) above target
  Normal   SuccessfulRescale            8m57s  horizontal-pod-autoscaler  New size: 10; reason: cpu r
esource utilization (percentage of request) above target
  Normal   SuccessfulRescale            26s    horizontal-pod-autoscaler  New size: 2; reason: All me
trics below target
```

Paso 9: Zero-Trust

gponce@dev: ~/taller-k8s-gponceb — 09-zero-trust

```
## ANTES de las NetworkPolicies (cualquier pod llega al backend)
$ kubectl run test-unauthorized --image=curlimages/curl --rm -i --restart=Never -q -- curl -s -o /dev/
null -w "HTTP %{http_code}
" --connect-timeout 3 http://backend-service:5678/api/productos; echo "exit=$?"
HTTP 200
exit=0

## Aplicando Zero-Trust
$ kubectl apply -f mod5-networking/05-default-deny-networkpolicy.yaml
networkpolicy.networking.k8s.io/default-deny-all created

$ kubectl apply -f mod5-networking/06-frontend-networkpolicy.yaml
networkpolicy.networking.k8s.io/allow-frontend created

$ kubectl apply -f mod5-networking/07-backend-networkpolicy.yaml
networkpolicy.networking.k8s.io/allow-backend created

$ kubectl apply -f mod5-networking/08-database-networkpolicy.yaml
networkpolicy.networking.k8s.io/allow-database created
networkpolicy.networking.k8s.io/allow-db-clients created

$ kubectl get networkpolicy
NAME               POD-SELECTOR       AGE
allow-backend      app=backend-api    5s
allow-database     app=postgres       5s
allow-db-clients   role=db-client     5s
allow-frontend     app=frontend-web   5s
default-deny-all   <none>             5s

## DESPUÉS: pod no autorizado -> backend (debe fallar)
$ kubectl run test-unauthorized --image=curlimages/curl --rm -i --restart=Never -q -- curl -s -o /dev/
null -w "HTTP %{http_code}
" --connect-timeout 3 http://backend-service:5678/api/productos; echo "exit=$?"
HTTP 000
pod gponceb/test-unauthorized terminated (Error)
exit=28

## DESPUÉS: pod no autorizado -> PostgreSQL por IP (sin DNS, debe hacer timeout)
$ kubectl run test-unauthorized-db --image=busybox:1.36 --rm -i --restart=Never -q -- sh -c 'nc -z -w
3 10.97.136.121 5432 && echo CONECTADO || echo BLOQUEADO'
BLOQUEADO

## DESPUÉS: pod con etiqueta app=frontend-web -> backend (autorizado)
$ kubectl run test-authorized --labels=app=frontend-web --image=curlimages/curl --rm -i --restart=Neve
r -q -- curl -s -o /dev/null -w 'HTTP %{http_code}
' --connect-timeout 3 http://backend-service:5678/api/productos
HTTP 200

## DESPUÉS: la app sigue funcionando a través del Ingress
$ curl -s -o /dev/null -w 'GET http://localhost/ -> HTTP %{http_code}
' http://localhost/
GET http://localhost/ -> HTTP 200

$ curl -s -o /dev/null -w 'GET http://localhost/api/productos -> HTTP %{http_code}
```

```
' http://localhost/api/productos
GET http://localhost/api/productos -> HTTP 200

## DESPUÉS: el backup (role=db-client) sigue llegando a PostgreSQL
$ kubectl create job backup-manual-2 --from=cronjob/db-backup-cronjob
job.batch/backup-manual-2 created

$ kubectl logs job/backup-manual-2
=== GENERANDO BACKUP EN VOLUMEN PERSISTENTE ===
Backup guardado exitosamente en: /backups/backup-20261023-150233.sql
```

7. Paso 10 — Matriz de troubleshooting

Escenario	Diagnóstico observado	Solución
ImagePullBackOff	`describe pod` → *lookup registro-inexistente.com: no such host*	`kubectl set image ... web=nginx:alpine` → Running
CrashLoopBackOff	`logs` → *FATAL: Variable de entorno DB_HOST no configurada*; Exit Code 1, reinicios crecientes	Definir la variable / corregir el comando del contenedor
OOMKilled	`describe pod` → Reason OOMKilled, Exit Code 137, límite 20Mi vs 50M solicitados	Subir `limits.memory` o reducir el consumo
PVC Pending	`describe pvc` → *storageclass "storage-class-inexistente" not found*	Usar una StorageClass existente (`local-path`)
Ingress 502/503	`curl /broken-route` → HTTP 503; logs del controller `[gponceb-backend-service-9999]`; el Service solo expone 5678	Corregir `port.number: 5678`
NetworkPolicy bloquea DNS	Con `allow-backend` presente NO bloquea (las políticas son aditivas). Al quedar sola: *connection timed out; no servers could be reached* y la API cae (HTTP 000)	Eliminar la política errónea y restaurar la de permisos

gponce@dev: ~/taller-k8s-gponceb — 10-troubleshooting

```
######## 1. ImagePullBackOff
$ kubectl apply -f mod8-troubleshooting/01-error-imagepullbackoff.yaml
deployment.apps/error-image-pull created

######## 2. CrashLoopBackOff
$ kubectl apply -f mod8-troubleshooting/02-error-crashloopbackoff.yaml
deployment.apps/error-crash-loop created

######## 3. OOMKilled
$ kubectl apply -f mod8-troubleshooting/03-error-oomkilled.yaml
deployment.apps/error-oom-killed created

######## 4. PVC Pending
$ kubectl apply -f mod8-troubleshooting/04-error-pvc-pending.yaml
persistentvolumeclaim/error-pvc-pending created

$ kubectl get pods -l 'app in (image-fail,crash-fail,oom-fail)'
NAME                               READY   STATUS             RESTARTS      AGE
error-crash-loop-6889fc9c5c-jv7sv   0/1     Error              3 (61s ago)   75s
error-image-pull-85794f5bb5-b2k5x   0/1     ImagePullBackOff   0             75s
error-oom-killed-5dcc86d84d-msn44   0/1     OOMKilled          3 (56s ago)   75s

######## 1. Diagnóstico ImagePullBackOff
$ kubectl describe pod -l app=image-fail | sed -n '/Events:/,$p' | grep -E 'Failed|BackOff' | head -4
  Warning  Failed     33s (x3 over 74s)  kubelet            spec.containers{web}: Failed to pull image
"registro-inexistente.com/app-ficticia:v999": failed to pull and unpack image "registro-inexistente.co
m/app-ficticia:v999": failed to resolve image: failed to do request: Head "https://registro-inexistent
e.com/v2/app-ficticia/manifests/v999": dial tcp: lookup registro-inexistente.com: no such host
  Warning  Failed     33s (x3 over 74s)  kubelet            spec.containers{web}: Error: ErrImagePull
  Normal   BackOff    5s (x5 over 73s)   kubelet            spec.containers{web}: Back-off pulling ima
ge "registro-inexistente.com/app-ficticia:v999"
  Warning  Failed     5s (x5 over 73s)   kubelet            spec.containers{web}: Error: ImagePullBack
Off

$ kubectl set image deployment/error-image-pull web=nginx:alpine
deployment.apps/error-image-pull image updated

$ kubectl get pods -l app=image-fail
```

```
NAME                                READY   STATUS        RESTARTS   AGE
error-image-pull-6bdd566765-2tfrv   1/1     Running       0          3s
error-image-pull-85794f5bb5-b2k5x   0/1     Terminating   0          79s

######## 2. Diagnóstico CrashLoopBackOff
# Nota: 'kubectl logs --previous' devolvió "unable to retrieve container logs" porque el kubelet ya
# había rotado el log del intento anterior; el contenedor actual ya terminó con el mismo error:
$ kubectl logs -l app=crash-fail | head -6
Error: FATAL: Variable de entorno DB_HOST no configurada
    at [eval]:1:7
    at runScriptInThisContext (node:internal/vm:143:10)
    at node:internal/process/execution:100:14
    at [eval]-wrapper:6:24
    at runScript (node:internal/process/execution:83:62)

$ kubectl describe pod -l app=crash-fail | grep -E 'State:|Reason:|Exit Code|Restart Count'
    State:          Terminated
      Reason:       Error
      Exit Code:    1
    Last State:     Terminated
      Reason:       Error
      Exit Code:    1
    Restart Count:  3

######## 3. Diagnóstico OOMKilled
$ kubectl describe pod -l app=oom-fail | grep -E 'Reason:|Exit Code|Limits:|memory:|Restart Count'
      Reason:       OOMKilled
      Exit Code:    137
      Reason:       OOMKilled
      Exit Code:    137
    Restart Count:  3
    Limits:
      memory:  20Mi
      memory:     20Mi

######## 4. Diagnóstico PVC Pending
$ kubectl get pvc error-pvc-pending
NAME                STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS                VOLUMEATT
RIBUTESCLASS   AGE
error-pvc-pending   Pending                                      storage-class-inexistente   <unset>
80s

$ kubectl describe pvc error-pvc-pending | sed -n '/Events:/,$p'
Events:
  Type     Reason              Age                From                         Message
  ----     ------              ----               ----                         -------
  Warning  ProvisioningFailed  11s (x6 over 80s)  persistentvolume-controller  storageclass.storage.k8
s.io "storage-class-inexistente" not found

######## 5. Ingress 502/503 Bad Gateway
$ kubectl apply -f mod8-troubleshooting/05-error-ingress-502-bad-gateway.yaml
ingress.networking.k8s.io/error-ingress-bad-gateway created

$ curl -s -o /dev/null -w 'GET http://localhost/broken-route -> HTTP %{http_code}
' http://localhost/broken-route
GET http://localhost/broken-route -> HTTP 503

$ kubectl get endpointslices -l kubernetes.io/service-name=backend-service
NAME                    ADDRESSTYPE   PORTS   ENDPOINTS                     AGE
backend-service-khg9c   IPv4          5678    10.244.90.144,10.244.90.145   35m

$ kubectl get svc backend-service -o jsonpath='{.spec.ports[*].port}{"
"}'
5678

$ kubectl -n ingress-nginx logs deploy/ingress-nginx-controller --tail=200 | grep -iE 'broken|9999' |
tail -3
10.51.20.21 - - [23/Oct/2026:15:04:57 +0000] "GET /broken-route HTTP/1.1" 503 190 "-" "curl/8.23.0" 85
0.000 [gponceb-backend-service-9999] [] - - - - d1b6a67312a5b4f395dadb978d592d14
```

```
######## 6. NetworkPolicy que bloquea DNS
$ kubectl exec deploy/backend-api -c api -- nslookup postgres-service
Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find postgres-service.cluster.local: NXDOMAIN


** server can't find postgres-service.svc.cluster.local: NXDOMAIN

Name:   postgres-service.gponceb.svc.cluster.local
Address: 10.97.136.121

** server can't find postgres-service.svc.cluster.local: NXDOMAIN

** server can't find postgres-service.cluster.local: NXDOMAIN

command terminated with exit code 1

$ kubectl apply -f mod8-troubleshooting/06-error-networkpolicy-blocked.yaml
networkpolicy.networking.k8s.io/error-block-dns created

$ kubectl exec deploy/backend-api -c api -- nslookup -timeout=3 postgres-service
Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find postgres-service.cluster.local: NXDOMAIN

** server can't find postgres-service.cluster.local: NXDOMAIN


Name:   postgres-service.gponceb.svc.cluster.local
Address: 10.97.136.121

** server can't find postgres-service.svc.cluster.local: NXDOMAIN

** server can't find postgres-service.svc.cluster.local: NXDOMAIN

command terminated with exit code 1

$ curl -s -m 10 -o /dev/null -w 'GET http://localhost/api/productos -> HTTP %{http_code}
' http://localhost/api/productos
GET http://localhost/api/productos -> HTTP 200

>> Observación: con allow-backend presente, error-block-dns NO bloquea el DNS (nslookup resuelve
>> postgres-service.gponceb.svc.cluster.local -> 10.97.136.121; el exit code 1 de nslookup se debe a
>> los dominios de búsqueda NXDOMAIN, no a un bloqueo). Las NetworkPolicies son ADITIVAS (unión OR):
>> una política con "egress: []" no revoca lo que otra política ya permite. El fallo aparece cuando
>> la única política que selecciona al pod es la restrictiva. Se simula eliminando allow-backend:

$ kubectl delete networkpolicy allow-backend
networkpolicy.networking.k8s.io "allow-backend" deleted from gponceb namespace

$ kubectl exec deploy/backend-api -c api -- nslookup -timeout=2 -retry=1 postgres-service 2>&1 | tail
-3
;; connection timed out; no servers could be reached

command terminated with exit code 1

$ curl -s -m 10 -o /dev/null -w 'GET http://localhost/api/productos -> HTTP %{http_code}
' http://localhost/api/productos
GET http://localhost/api/productos -> HTTP 000

$ kubectl get pods -l app=backend-api
NAME                           READY   STATUS    RESTARTS   AGE
backend-api-7f8496c7f9-68znx   2/2     Running   0          19m
backend-api-7f8496c7f9-rbtbx   2/2     Running   0          19m
```

```
>> Solución: eliminar la política errónea y restaurar allow-backend
$ kubectl delete networkpolicy error-block-dns
networkpolicy.networking.k8s.io "error-block-dns" deleted from gponceb namespace

$ kubectl apply -f mod5-networking/07-backend-networkpolicy.yaml
networkpolicy.networking.k8s.io/allow-backend created

$ kubectl exec deploy/backend-api -c api -- nslookup postgres-service 2>&1 | grep -A1 '^Name'
Name:   postgres-service.gponceb.svc.cluster.local
Address: 10.97.136.121

$ curl -s -m 10 -o /dev/null -w 'GET http://localhost/api/productos -> HTTP %{http_code}
' http://localhost/api/productos
GET http://localhost/api/productos -> HTTP 200

>> Limpieza de los escenarios de error
$ kubectl delete -f mod8-troubleshooting/01-error-imagepullbackoff.yaml -f mod8-troubleshooting/02-err
or-crashloopbackoff.yaml -f mod8-troubleshooting/03-error-oomkilled.yaml -f mod8-troubleshooting/04-er
ror-pvc-pending.yaml -f mod8-troubleshooting/05-error-ingress-502-bad-gateway.yaml
deployment.apps "error-image-pull" deleted from gponceb namespace
deployment.apps "error-crash-loop" deleted from gponceb namespace
deployment.apps "error-oom-killed" deleted from gponceb namespace
persistentvolumeclaim "error-pvc-pending" deleted from gponceb namespace
ingress.networking.k8s.io "error-ingress-bad-gateway" deleted from gponceb namespace
```

8. Conclusiones

- La guía funciona conceptualmente, pero en un clúster real requiere CNI con soporte de NetworkPolicy, un aprovisionador de almacenamiento, metrics-server y un Ingress Controller expuesto; en Docker Desktop varios de estos vienen implícitos.
- En Zero-Trust no basta con abrir el tráfico entre aplicaciones: también hay que permitir DNS, el Ingress Controller y los clientes batch (Jobs/CronJobs) que acceden a la base de datos.
- Las NetworkPolicies son aditivas: una política restrictiva no anula lo que otra ya permite (escenario 6).
- Las ResourceQuotas acotan el escalado del HPA; deben dimensionarse junto con `maxReplicas` y los `limits` por pod.